Electronic Development Project Report

RFID Digital Jukebox with M5Stack CoreS3

Digital Systems — Bachelor's Degree in Sound and Image Engineering in Telecommunications
University of Extremadura (UEx)
Author: Daniel Santillana

Table of Contents

1. Brief Description (Abstract)
2. Motivation
3. State of the Art
4. Techniques and Materials Used
5. Schematics, Diagrams, Photos, Files, or Code
6. GitHub Repository
7. Results Obtained
8. Future Developments
9. Conclusion
10. Sources Used

1. Brief Description (Abstract)

The project consists of the design and development of an interactive music player (Jukebox) based on the ESP32-S3 microcontroller (integrated within an M5Stack CoreS3 SE). The system enables audio playback by reading RFID/NFC cards associated with different music albums. Upon scanning a card, the device's LCD display renders the album cover art (loaded from an internal microSD card) and plays the corresponding audio track via a DFPlayer Mini decoder module, amplifying the signal through a Class-D audio amplifier (PAM8403) to external speakers.

2. Motivation

The mass digitization of music consumption through streaming platforms has diluted the tactile experience associated with music. In recent years, however, the public has reaffirmed the value of analog media, triggering a surge in vinyl record sales. This project stems from my motivation to recover the ritual of "putting on a record," combining the nostalgia of tangible objects with the versatility of current embedded systems. My goal in developing this prototype is to offer an interactive cataloging system, useful in both home environments and record stores, allowing users to dynamically explore an album's complete information prior to purchase. Furthermore, it provided an excellent practical framework to apply course concepts, integrating communication buses (SPI, UART), power management, and audio processing.

3. State of the Art

Commercial products utilizing similar concepts currently exist for children, such as "Toniebox" or "Yoto Player," which use NFC figurines or cards to trigger story and song playback. In the maker and DIY community, Raspberry Pi-based players (such as the Phoniebox project) are common; however, they typically require a full operating system (Linux) with slow boot times. Using RTOS microcontrollers (such as the ESP32) represents a significantly more efficient, instant-on, and low-power alternative. Furthermore, the integrated use of these microcontrollers within the M5Stack product lineup allows the system complexity to scale according to user needs while delivering a much more professional appearance.

4. Techniques and Materials Used

The following table details the materials used in the development of the project:

Component / Material	Description / Function	Qty	Approx. Cost	Supplier / Notes
M5Stack CoreS3 SE	Main ESP32-S3 MCU, capacitive touch display, AXP2101 PMIC	1	€40	

MFRC522 Reader Module	RFID/NFC Reader (SPI bus communication) for album detection	1	€2
DFPlayer Mini	MP3 Decoder & microSD reader (UART communication)	1	€2.10
PAM8403 Amplifier	Class-D Audio Amplifier (2x3W) for analog signal	1	€2
MicroSD Card	32GB memory (FAT32) for storing audio tracks	1	€7
MicroSD Card	16GB memory (FAT32) for storing cover artwork	1	€5
RFID Cards / Tags	13.56MHz passive cards for the music catalog	4	€2
Dupont Cables	Connection wiring and signal adaptation (resistors as needed)	35	€3

Development Techniques and Protocols:

- Finite State Machine (FSM) for software flow control (MODO_ESPERA / STANDBY_MODE, MOSTRANDO_CARATULA / DISPLAYING_COVER, MODO_HISTORIAL / HISTORY_MODE).
- Shared SPI Bus between the internal SD card reader and the RC522, independently managing Chip Select (CS) lines.
- Asynchronous UART Bus for sending hexadecimal playback and volume commands to the DFPlayer Mini. Hardware troubleshooting was applied by removing the 1kΩ resistor from the TX bus to accommodate native 3.3V logic tolerances.
- Active Power Management (PMIC): Software routines (M5.Power.setVext(true)) to enable regulated 5V power outputs to external peripheral modules.

5. Schematics, Diagrams, Photos, Files, or Code

Key Wiring Diagram developed during physical testing:

1. Power Supply (Port A - Red Grove Port):

- Pin 5V (VEXT) -> VCC on DFPlayer and VCC on PAM8403
- Pin GND -> GND on DFPlayer and GND on PAM8403

2. UART Data (Port C - Blue Grove Port):

- GPIO 17 (TX) -> RX on DFPlayer (direct 3.3V connection)
- GPIO 18 (RX) -> TX on DFPlayer

3. Analog Audio (Zero differential DAC routing):

- DAC_L and DAC_R on DFPlayer -> L and R inputs on PAM8403 (avoiding amplified SPK outputs to prevent distortion)

Logical Code Structure (Main Loop Snippet):

```
void loop() {
  M5.update();
  gestionarPantallaTactil();
  gestionarTemporizadores();
  if (estadoActual == MODO_ESPERA) {
    gestionarRFID();
  }
  delay(5);
}
```

6. GitHub Repository

The complete project documentation, schematics, and C++ source code are available at the following GitHub repository:

https://github.com/saasser/Jukebox-RFID-M5Stack

7. Results Obtained

A stable and fully functional system with instant boot times was successfully achieved. The M5Stack reads RFID cards with zero false positives, extracts graphic cover art from the internal SD card using the M5Unified library, and renders it on screen. Simultaneously, the DFPlayer module flawlessly interprets UART commands (after tuning the pins to Port C and native 3.3V logic) and feeds a clean analog signal to the PAM8403 amplifier, which plays the albums at an optimal volume level controlled via software (volume set to 22/30).

8. Future Developments

Subsequent iterations of the project could include:

- Expanded Touch Interface: Adding on-screen capacitive buttons on the M5Stack LCD to dynamically pause, resume, and adjust volume.
- Dynamic Database: Implementing WiFi connectivity (native to ESP32) to fetch cover artwork from a REST API (e.g., Spotify API) and store cloud playback logs instead of relying solely on local SD storage.
- Enclosure Design: 3D modeling and printing a retro or minimalist jukebox chassis to house the hardware, speaker acoustics, and top-mounted RFID reader.

9. Conclusion

The project proved to be a deeply rewarding and satisfying experience despite initial development hurdles. Key challenges included brownout resets when powering on the M5Stack due to host computer

USB port current limits, integrating the internal microSD cards on the M5Stack, and setting up UART communication with the DFPlayer Mini. Overall, the project was a technical and educational success. It demonstrated that integrating heterogeneous peripherals (SPI, UART, Analog Audio) demands strict attention to voltage tolerances and bus multiplexing. Overcoming obstacles such as memory bus interference, logic level drops caused by unnecessary series resistors, or power cutoffs managed by the internal PMIC (AXP2101) served to consolidate essential skills in embedded hardware debugging and physical layer analysis.

10. Sources Used

- Official M5Stack documentation (m5stack.com) for operation and pinout details of the M5Stack CoreS3 SE.
- The Gemini AI assistant was consulted for code troubleshooting, interpreting M5Stack pin datasheets, and configuring the PAM8403 amplifier correctly.
- Approximate number of queries (prompts): ~15 interactive queries during the debugging phase.

Answers to Required AI Questions:

- Energy Cost Estimation of Queries: Assuming an industry average estimate of ~3 Wh per complex inference query (multimodal LLM) due to server computation, 15 queries represent an estimated energy consumption of approximately 45 Wh (0.045 kWh) during project development.
- AI Training Cost (Financial and Energy): Although exact foundational training data for Gemini is proprietary, state-of-the-art models with hundreds of billions of parameters require thousands of TPU accelerators operating over several months. Industry estimates place the training cost of GPT-4 or Gemini scale models at around $50–100 million USD and a massive energy footprint of approximately 50,000 MWh (50 GWh) per full training run.